FreiCAR
Practical Autonomous Driving

Team Name: **DiveDeep**

Exercise 2 - **2D Object Detection**

Repository:
https://github.com/madhu-basavanna/Freicar_DiveDeep/tree/master/02-01-object-detection-exercise

Ajesh Krishnan Kizhakke Menakath(5053372)
Kiran Kumaraswamy(4964003)
Madhu Basavanna(5053269)
Prasad Ramachandra Lakshmi(4971677)

2 Tasks

2.1 Complete Object Detection Codebase

- Images were read using the given loadRGB() and the image tensor and BBS values are stored in a dictionary.
- Data was trained and evaluated on EfficientDet

2.2 Model Evaluation

Data Augmentation

NOTE: Since we trained the models augmenting all the data with high probability especially random flip, we got poor performance for data with augmentation. By the time time we realized the model is performing poor with augmentation, it was too late to retrain and get good performance with better augmentation methods.

We trained the models with data augmented using the following transformation methods of TorchVision transforms as we didn't know that we had to implement our own augmentation. However, we have implemented our own implementation of data augmentation which we haven't trained.

These are the augmentations we implemented

1. Brightness change with jitter value uniformly sampled from the range [0, 3] with probability 0.46
2. Horizontal Flip with a probability of 0.02(Bounding box values are updated respectively)
3. Both brightness change and Horizontal Flip with probability 0.02
4. Images are kept as it is (without augmentation) with a probability 0.5

Below are the data augmentation techniques used for training

1. Horizontal Flip with a probability of 0.05
2. ColorJitter with brightness, contrast, saturation and hue changes with jitter value uniformly sampled from the range [0.1, 0.5] with probability 0.1
3. GaussianBlur with kernel size 31x31 and sigma value sampled uniformly from the range [0.1, 0.5] with probability 0.15
4. Both ColorJitter and GaussianBlur with probability 0.2
5. Images are kept as it is (without augmentation) with a probability 0.5

	Data without augmentation		Data with augmentation
Real Data	• Class. Loss : 0.04427 • Reg. Loss : 0.34020 • Total Loss : 0.38447 • Epochs : 20 • Batch Size : 4	• Class. Loss : 0.02890 • Reg. Loss : 0.22238 • Total Loss : 0.25128 • Epochs : 39 • Batch Size : 4	• Class. Loss : 0.09582 • Reg. Loss : 0.60890 • Total Loss : 0.70472 • Epochs : 20 • Batch Size : 7
Synth Data	• Class. Loss : 0.0987 • Reg. Loss : 0.7925 • Total Loss : 0.89135 • Epochs : 20 • Batch Size : 7	• Class. Loss : 0.02657 • Reg. Loss : 0.21499 • Total Loss : 0.24156 • Epochs : 80 • Batch Size : 7	• Class. Loss : 0.11485 • Reg. Loss : 0.88103 • Total Loss : 0.99588 • Epochs : 20 • Batch Size : 4

Performance on validation set

- We couldn't train all the models for all the cases more than 50 epochs due to time and resource constraints. Inorder to get equal evaluation metrics, we limited the comparisons to 20 epochs. Hence we could see less performance in the models in some cases. However, we have also added cases where we have evaluated for more than 50 epochs in the table. **We can see good performance improvement for more epochs.**
- We have used

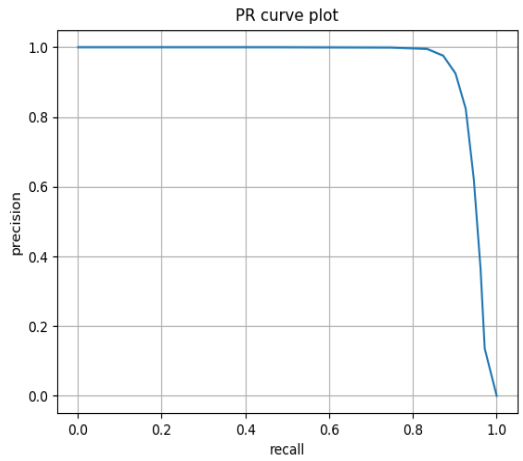
Below are the mIoU values calculated with IoU threshold = 0.5 and threshold = 0.2

	Data without augmentation	Data with augmentation
Real Data	• mIoU : 0.7462 (20 epochs)	• mIoU : 0.5838 (20 epochs)
Synth Data	• mIoU : 0.8193 (20 epochs) • **mIoU : 0.9030 (80 epochs)**	• mIoU : 0.7873 (20 epochs)

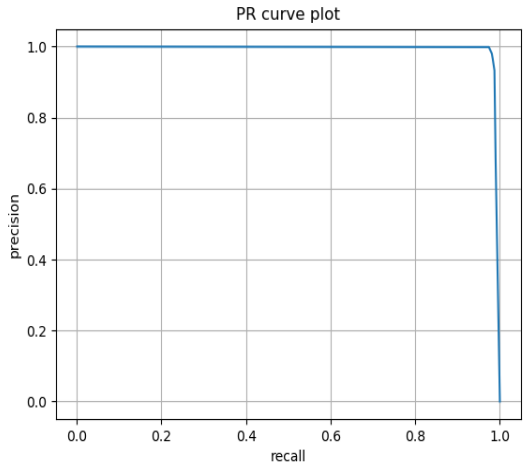
Below are the mAP values calculated with IoU threshold 0.5 and threshold value is varied between 0 & 1 with an interval 0.1. We calculated the area under the curve which is also the average precision(AP) for one class. Since we have only one class, this is also our mAP.

	Data without Data without augmentation	Data with augmentation
Real Data	• mAP : 0.7715 (20 epochs)	• mAP : 0.6699(20 epochs)

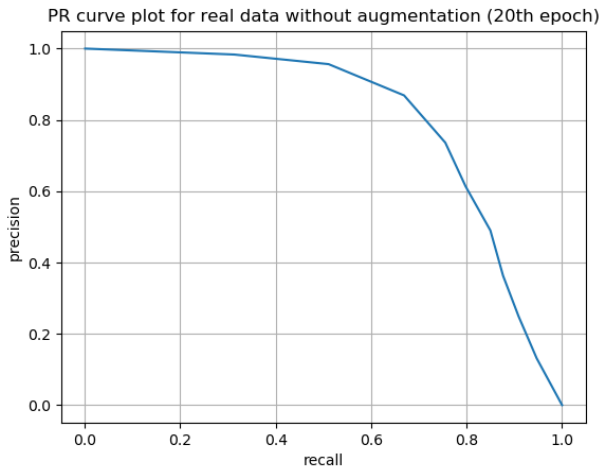
Synth Data	• mAP : 0.93653 (20 epochs) • **mAP : 0.9865 (80 epochs)**	• mAP : 0.8679(20 epochs)



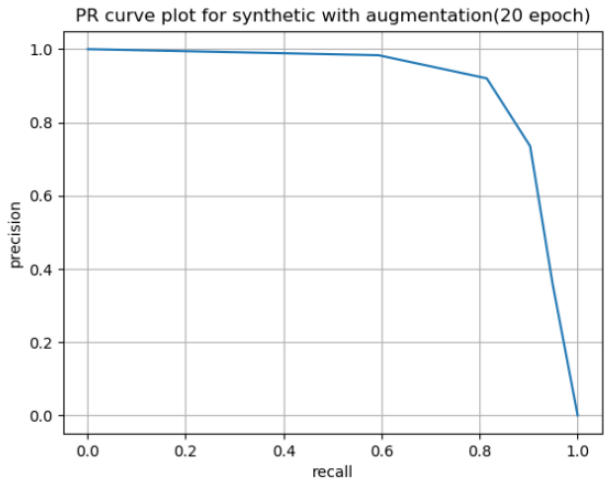
Synth-data without augmentation - 20 epochs



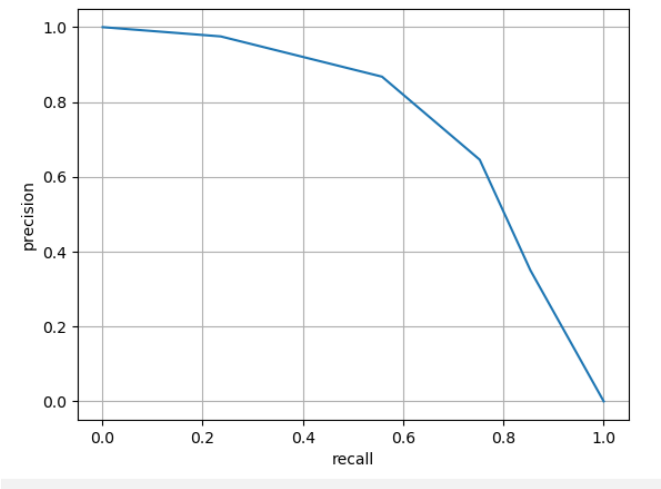
Synth-data without augmentation - **80 epochs**



Real-data without augmentation - 20 epochs



Synth-data with augmentation - 20 epochs	



Real-data with augmentation - 20 epochs	

FPS Analysis

All models are considered for 20 epochs

	Data without augmentation	Data with augmentation
Real Data	● Time : 0.0356 sec ● FPS : 28.025	● Time : 0.030 sec ● FPS : 32.3287
Synth Data	● Time : 0.03942 sec ● FPS : 25.3636	● Time : 0.0366 sec ● FPS : 27.2680

2.3 ROS Node

- Take a clone of the 02-01-object-detection-exercise repository from the link mentioned above
- Run catkin build from terminal at file location ~/freicar_ws you will find the build of 'perception_pkg' package under ROS folder
- Run '*roslaunch freicar_launch local_comp_launch.launch*' from the terminal to launch the simulator
- Run 'r*oslaunch freicar_agent sim_agent.launch name:=freicar_anyname tf_name:=freicar_anyname spawn/x:=0 spawn/y:=0 spawn/z:=0 spawn/heading:=20 use_yaml_spawn:=true sync_topic:=!'* to spawn the car
- Run *python inference.py -w /path/to/your/weights.pth*
- The bounding box information is published from node 'Image_bbs' to topic 'predicted_bbs'
- The Image_bbs node is also subscribed to '*/freicar_anyname/sim/camera/rgb/front/image*' and converting it to cv2 format.